Grammar-Aware Fuzzing of PostgreSQL, and Beyond

CS-5187 Software Analysis, Term Project A3

Venu Kummari Bhuvan Chilukuti Gowtham Reddy

November 2026

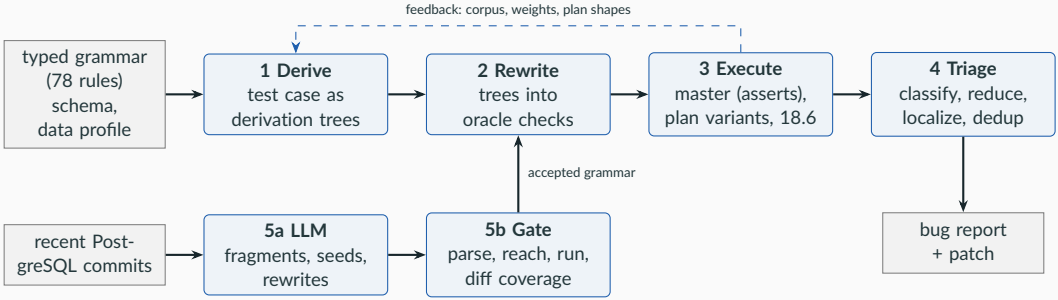
IIT Guwahati

Problem: the oracle for wrong results

- Target: PostgreSQL, master built with **assertions**, compared with release 18.6.
- Four failure classes: crash/assertion, **wrong result**, internal error, hang.
- A wrong result leaves no trace: the tool needs a test oracle that knows the right answer.

Idea
Generate SQL as typed derivation trees; rewrite each tree into equivalent queries and run it under other plans and on the last release. Any disagreement is a bug.

Architecture of the PostgreSQL fuzzer



- One derivation tree, four jobs: generate, mutate, rewrite into oracles, reduce.
- 10,500 lines of Python, 151 unit tests.

Techniques: grammar and oracles

Typed grammar (own DSL, 78 rules)

- types as parameters, scope-aware names ⇒ 99.3% valid statements (random mutation 26%)
- feedback: corpus, subtree mutation, bandit weights, data-aware literals

Oracle stack

- server watches: crash/assertion (server log), internal error `XX000`, hang (timeout and cancel ignored)
- **plan variants**: 18 planner settings, same answer required
- **metamorphic tree rewrites**: TLP $Q = Q_p \uplus Q_{\neg p} \uplus Q_{p\ \mathrm{IS\ NULL}}$, NoREC, $e \to (\mathrm{SELECT}\ e)$
- **cross-version**: master vs 18.6
- **CERT**: a restriction must not raise the row estimate

Techniques: triage and the LLM gate

Triage

1. reproduce $\times 3$ on a fresh server
2. reduce on the tree (smallest derivation of the same type), then statements, then tokens: 116 $\rightarrow$ 79 tokens
3. localize: plan variant, rewrite or version
4. deduplicate by signature

LLM proposes, server decides

- reads a recent commit, proposes grammar, seeds, rewrites
- gate: parses, reachable, $\geq 60\%$ runs, executes the commit's changed lines (gcov)
- 4 of 10 commits passed

How the tool found the PostgreSQL bugs

Bug type	Found by	Signal	Upstream
assertion	LLM fragment + TLP check	assertion during a metamorphic check	**fixed** #19803
wrong output	cross-version oracle	master and 18.6 disagreed	**fixed, our patch**
assertion	extended-statistics generator	assertion in the estimator	**fixed, our patch**
assertion / wrong result	grammar fuzzer + cross-version	assertion on master, wrong answer on 18.6	**fixed, our patch**
wrong result	cross-version + TLP	extra row on master	**fixed, our patch**
hang	hang oracle	timeout and cancel ignored	**fixed, our patch**
internal error	`XX000` oracle	“should not happen” error	**confirmed**
wrong result	$e \rightarrow (\mathrm{SELECT}\ e)$ rewrite	sub-select blocked a simplification	**open**
bad estimate	CERT oracle	estimate rose under a restriction	**open**

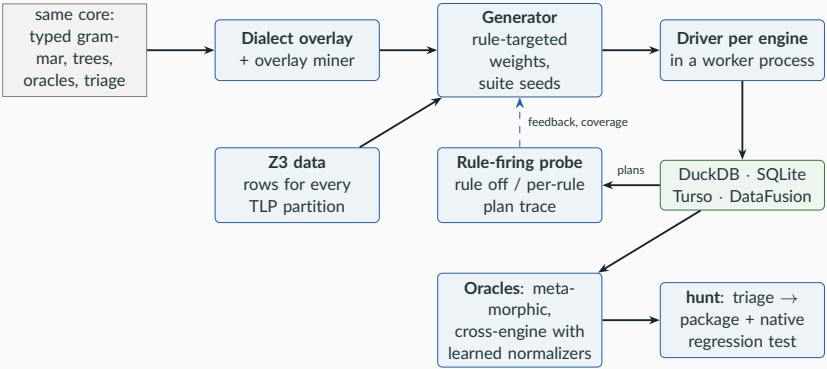
9 reported, 6 fixed upstream (5 with our patches). `scripts/reproduce.sh`: all 9 rediscovered in 150 s.

Results

RQ1 reach	backend lines **36.7% vs 22.2%** for random mutation ($\hat{A}_{12} = 1.00$, $p = 0.008$); oracles: plan shapes $\times 1.7$
RQ2 precision	93 candidates: 8 real, 15 duplicates, 42 false positives (all classes fixed), 28 flaky
RQ3 benchmark	23 real bugs re-injected; best arm found 2 of 20 in 1 h
RQ4 LLM	+5 SQL features, no coverage gain

Lesson: oracle precision, not generator reach, limits a wrong-result fuzzer.

Beyond PostgreSQL: the generalized fuzzer



- DuckDB, SQLite, Turso, DataFusion; same trees, oracles and triage.

Generalized techniques

- **Dialect overlays** + **overlay miner**: failing grammar alternatives become `%drop` lines (DuckDB 32% → 44% valid with no manual line; with overlays 79–98% valid).
- **Optimizer-rule firing** instead of code coverage: a rule fires if the plan changes when it is switched off (SQLite test control, DuckDB `disabled_optimizers`, DataFusion `EXPLAIN VERBOSE`); used as feedback, coverage and for rule-targeted generation.
- **Z3 data synthesis**: rows that make every TLP partition non-empty.
- **Learned normalizers** for cross-engine comparison: disagreements 28.7% → 3.6%.
- **Autonomous loop**: campaign → triage → package with a regression test in the engine's own format.

How the generalized fuzzer found its bugs

Engine	Bug type	Found by	State
Turso	wrong result	Turso-vs-SQLite differential oracle	**filed with fix**: #11594 / PR #11595
DuckDB	internal error (regression)	internal-error oracle on the nightly build	**filed with fix**: #28551 / PR #28552
DataFusion	internal error in DML	internal-error oracle	**filed with fix**: #27235 / PR #27236
3 more	panic, errors	worker caught a panic / error	fixed upstream before we filed

Each fix has a regression test in the engine's format and passes the engine's full suite.

Conclusion

9
PostgreSQL bugs reported

6
fixed upstream (5 our patches)

3
fixes filed to DuckDB, Turso, DataFusion

Team: Venu (tool, paper, generalized fuzzer) · Bhuvan (RQ3, bug hunts, four patches) · Gowtham (campaigns, LLM arm, two reports, one fix).

Thank you. Questions?

github.com/PASS-IITG/cs5187-term-project-blazing-bhuvi